Algorithms & Data Structures

Week 07

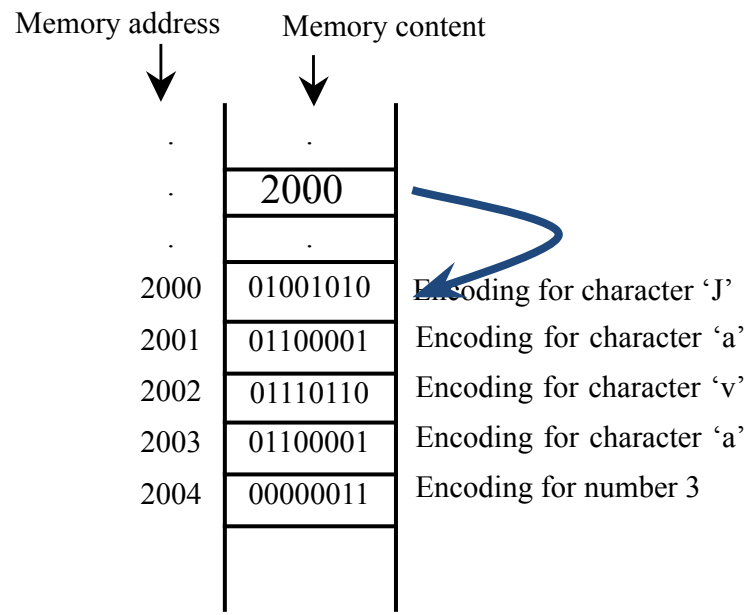
Ahmet Feyzi Ateş

General Information (2nd Phase)

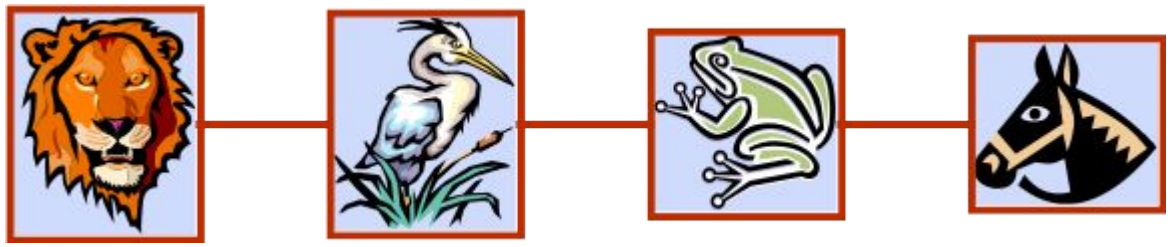
- Data, Memory, and CPU Relationship (revisited)
- Asymptotic Analysis and Big-O (revisited)
- Arrays and Dynamic Arrays (Array Lists)
- Single, Double, and Circular Linked Lists
- Stacks and Queues
- Priority Queues
- Hashing, Hash Tables, and Hash Maps
- Binary Trees and Binary Search Trees
- Minimum Heap and Maximum Heap
- Introduction to AVL and Red-Black Trees
- ***Choosing the Right Data Structure in Real-World Problems***

Pointers

- Each program variable is stored in the computer's memory at some location, or ***address***.
- A ***pointer*** is a variable that holds the value of such an address.

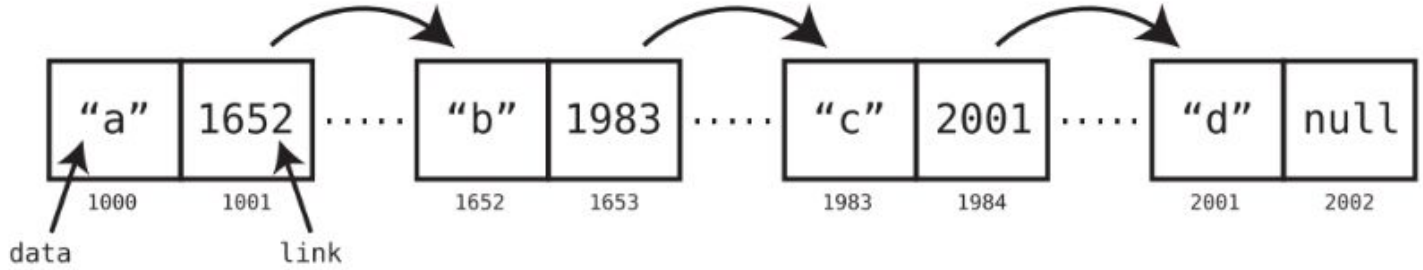


Linked Lists



- Last week, we presented the array data structure and discussed some of its applications.
- Arrays are nice and simple for storing things in a certain order, but they have drawbacks.
  - They are not very adaptable.
  - For instance, we have to fix the size n of an array in advance, which makes resizing an array difficult.
  - Insertions and deletions are difficult because elements need to be shifted around to make space for insertion or to fill empty positions after deletion.

Linked Lists



- Linked lists, on the other hand, work quite differently.
- Instead of being block of memory, the data from linked lists can be scattered across different cells throughout the computer's memory.
- Connected data that is dispersed throughout memory are known as ***nodes***.
- In a linked list, each node represents one item in the list.
- A linked list, in its simplest form, is a collection of ***nodes*** that together form a linear ordering.
- Each node, in addition to storing its associated element, also stores a pointer, called *next*, to the next node of the list.

Node in programming languages

Java

```java
public class Node {
    public int element;
    public Node next;
}
```

```java
public class Node {
    public String element;
    public Node next;
}
```

```java
public class Node<E> {
    public E element;
    public Node<E> next;
}
```

C++

```cpp
class StringNode {
    string elem;
    StringNode* next;
};
```

```cpp
class intNode {
    int elem;
    intNode* next;
};
```

```cpp
template <typename E>
class Node {
    E elem;
    Node<E>* next;
};
```

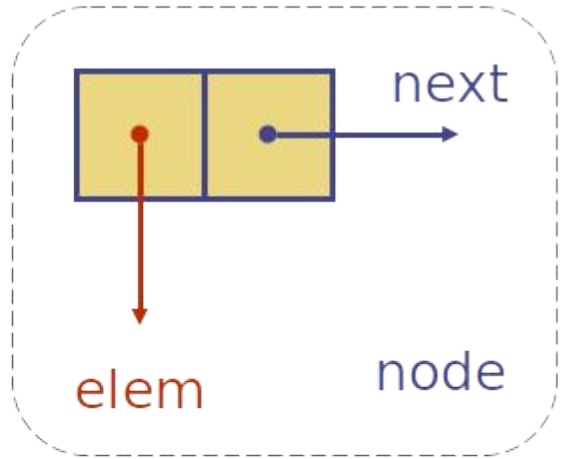
Python

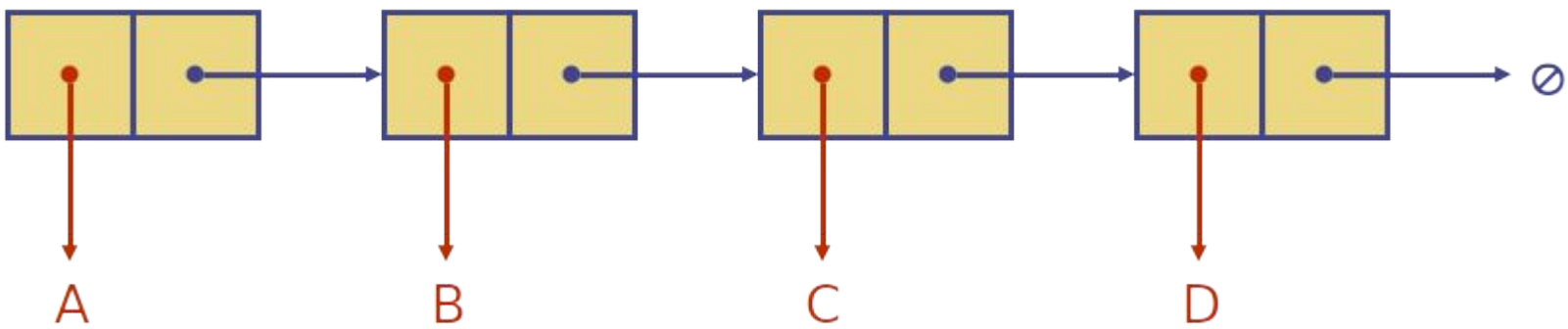
Python does not require an explicit type declaration for the `element` and `next` fields

```python
class Node:
    def __init__(self, element, next):
        self.element = element
        self.next = next
```

(Singly) Linked Lists

- A singly linked list is a concrete data structure consisting of a sequence of nodes
- Each node stores
  - element
  - link to the next node





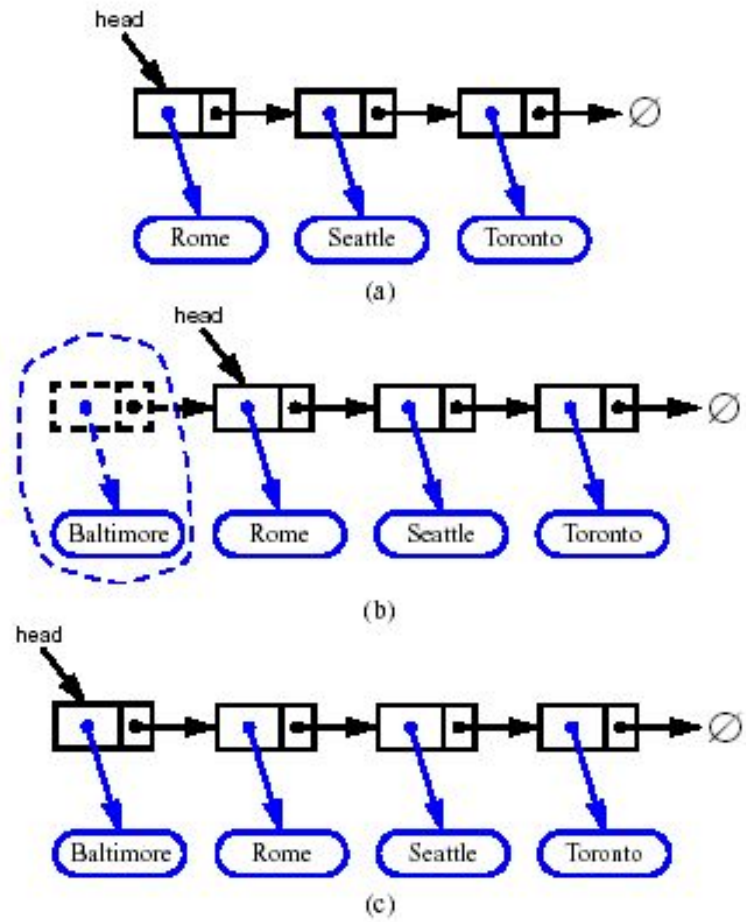
(Singly) Linked Lists



- The first and last nodes of a linked list are called the ***head*** and ***tail*** of the list, respectively.
- We can identify the tail as the node having a null next reference.
- The structure is called a singly linked list because each node stores a single link.
- Moving from one node to another by following a next reference is known as ***link hopping*** or ***pointer hopping***.
- Like an array, a singly linked list maintains its elements in a certain order, as determined by the chain of next links.
- Unlike an array, a singly linked list does not have a predetermined fixed size. It can be resized by adding or removing nodes.

Inserting at the Head

1. Allocate a new node
2. Insert new element
3. Have new node point to old head
4. Update head to point to new node



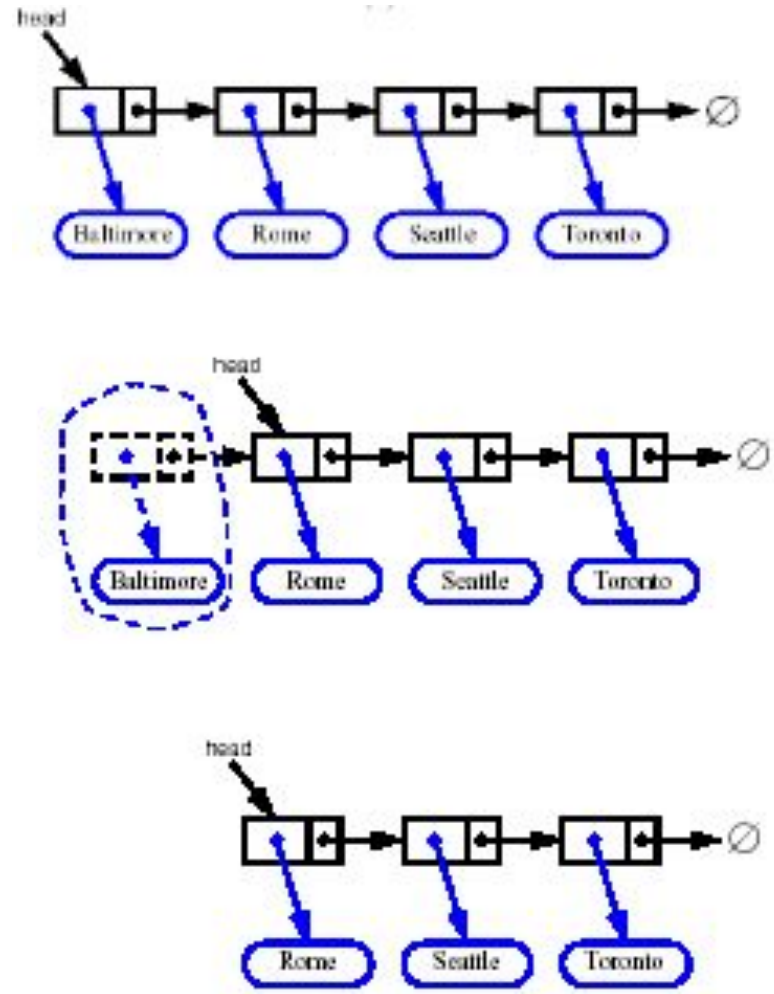
Algorithm for Inserting at the Head

Algorithm *insertAtHead*(e):

```
v ←new Node()          // Create a new node v
v.element ← e          // Insert new element
v.next ← head          // have new node point to old head.
head ← v               // update head to point new node.
```

Removing from the Head

1. Update head to point to next node in the list
2. Allow garbage collector to reclaim the former first node



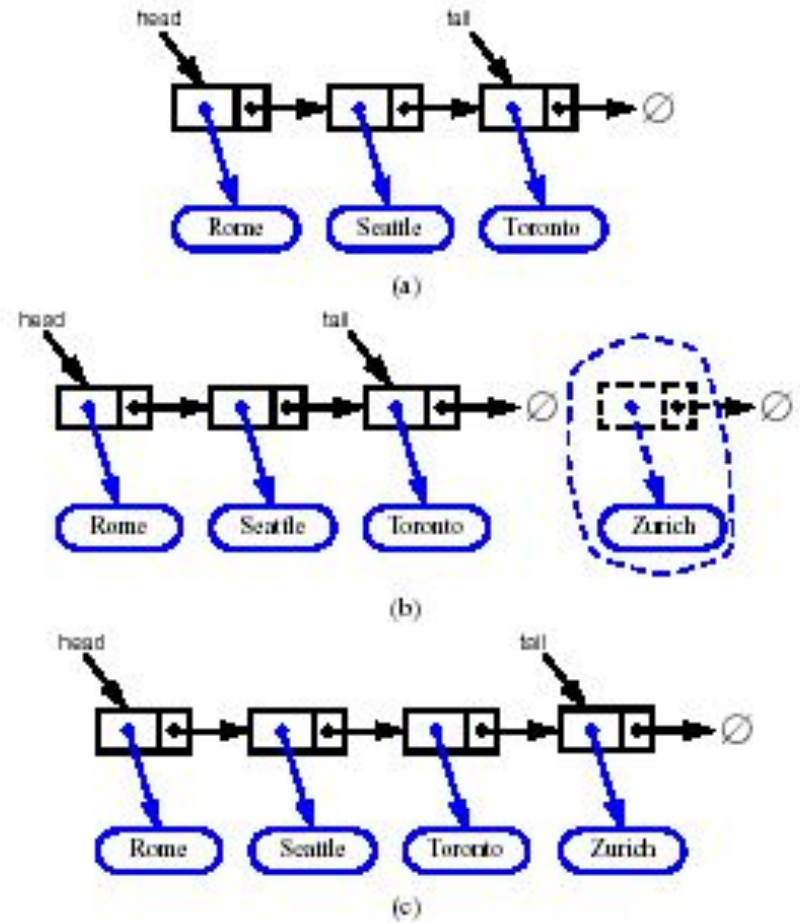
Algorithm for Removing from the Head

Algorithm *RemoveFromHead*():

```
u ← head              // Save old head temporarily.
head ← head.next      // Have head point to the old first element next.
delete (u)            // Delete the node pointed to by old head (u)
                      // Neither required nor possible in Java or Python
```

Inserting at the Tail

1. Allocate a new node
2. Insert new element
3. Have new node point to null
4. Have old last node point to new node
5. Update tail to point to new node



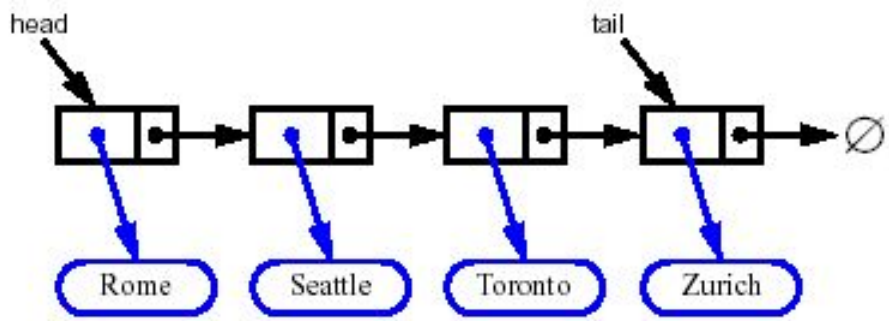
Algorithm for Inserting at the Tail

Algorithm *InsertAtTail*(e):

```
v ←new Node()          // Create a new node v
v.element ← e          // Insert new element
v.next ←∅              // Have new node point to null.
tail.next ←v           // Have old last node point to new node.
tail ← v               // Have tail point to the new element.
```

Removing at the Tail

- Removing at the tail of a singly linked list is not efficient!
- There is no constant-time way to update the tail to point to the previous node



Algorithm for Removing from the Tail

Algorithm *RemoveFromTail*():

```
// Search from the beginning for the next to last element
p ← head
while p.next != ∅ do
    temp ←p
    p ← p.next
delete (p)          // Neither required nor possible in Java or Python.
tail ←temp
tail.next ← ∅
```

Complexities

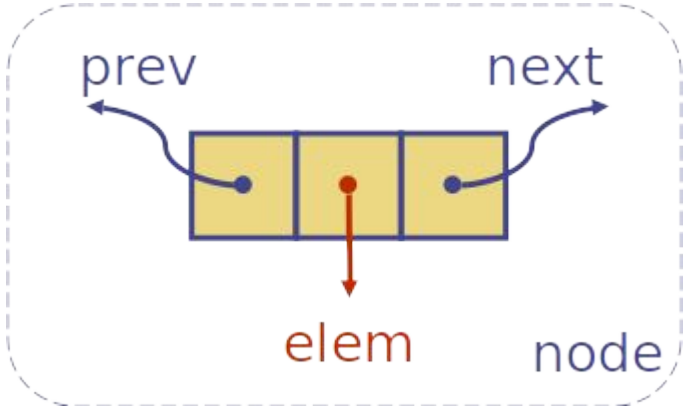
- In the implementation of the singly linked list.
- Assuming that two pointers (head and tail) are maintained:
  - The space used by each position of the list is O(1).
  - The space used by a list with n elements is O(n).
  - All the operations, except "*Removing from Tail*" run in O(1) time.
  - "*Removing from Tail*" runs in O(n).

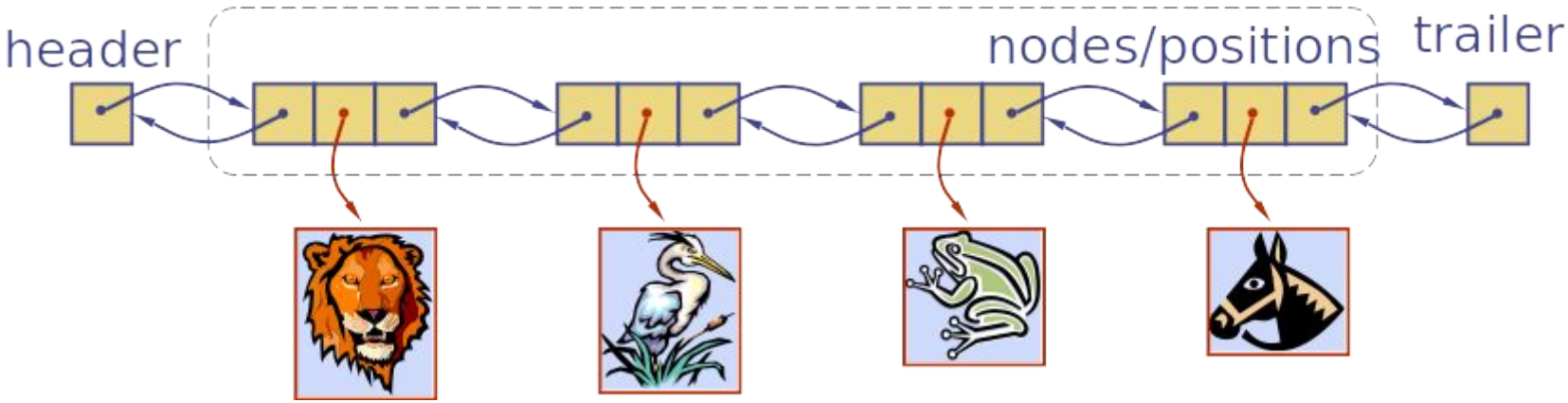
Doubly Linked List

- As we saw in the previous section, removing an element at the tail of a singly linked list is not easy.
- Indeed, it is time consuming to remove any node other than the head in a singly linked list,
  - since we do not have a quick way of accessing the node immediately preceding the one we want to remove.
- There is a type of linked list that allows us to go in both directions, forward and reverse, in a linked list.
  - It is the ***doubly linked list***.
- In addition to its element member, a node in a doubly linked list stores two pointers, a *next* link and a *prev* link, which point to the next node in the list and the previous node in the list, respectively.

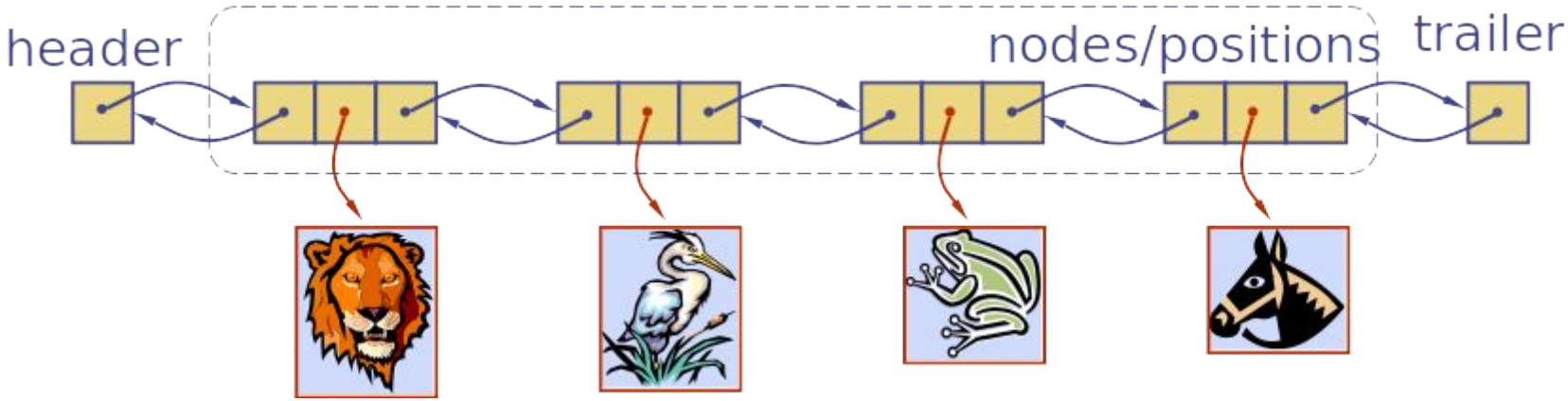
Doubly Linked List

- A doubly linked list provides a natural implementation of the Node List ADT
- Nodes implement Position and store:
  - element
  - link to the previous node
  - link to the next node
- Special trailer and header nodes





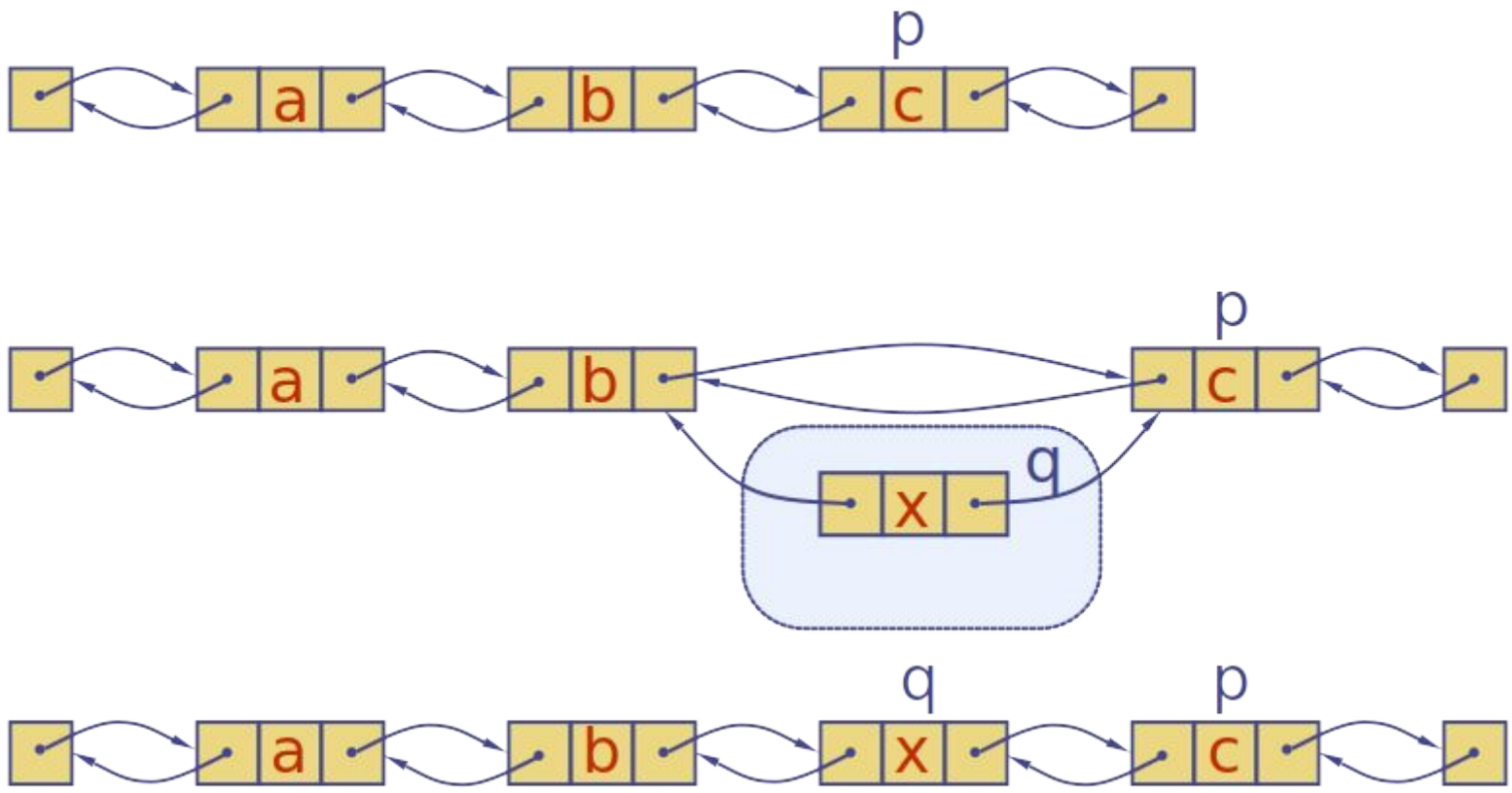
Doubly Linked List



- To simplify programming, it is convenient to add special nodes at both ends of a doubly linked list:
  - a header node just before the head of the list,
  - and a trailer node just after the tail of the list.
- These “dummy” or sentinel nodes do not store any elements.
- They provide quick access to the first and last nodes of the list.
- In particular, the header’s next pointer points to the first node of the list, and the prev pointer of the trailer node points to the last node of the list.

Insertion into Doubly Linked List

- We visualize operation insert(p, x), which inserts x before p

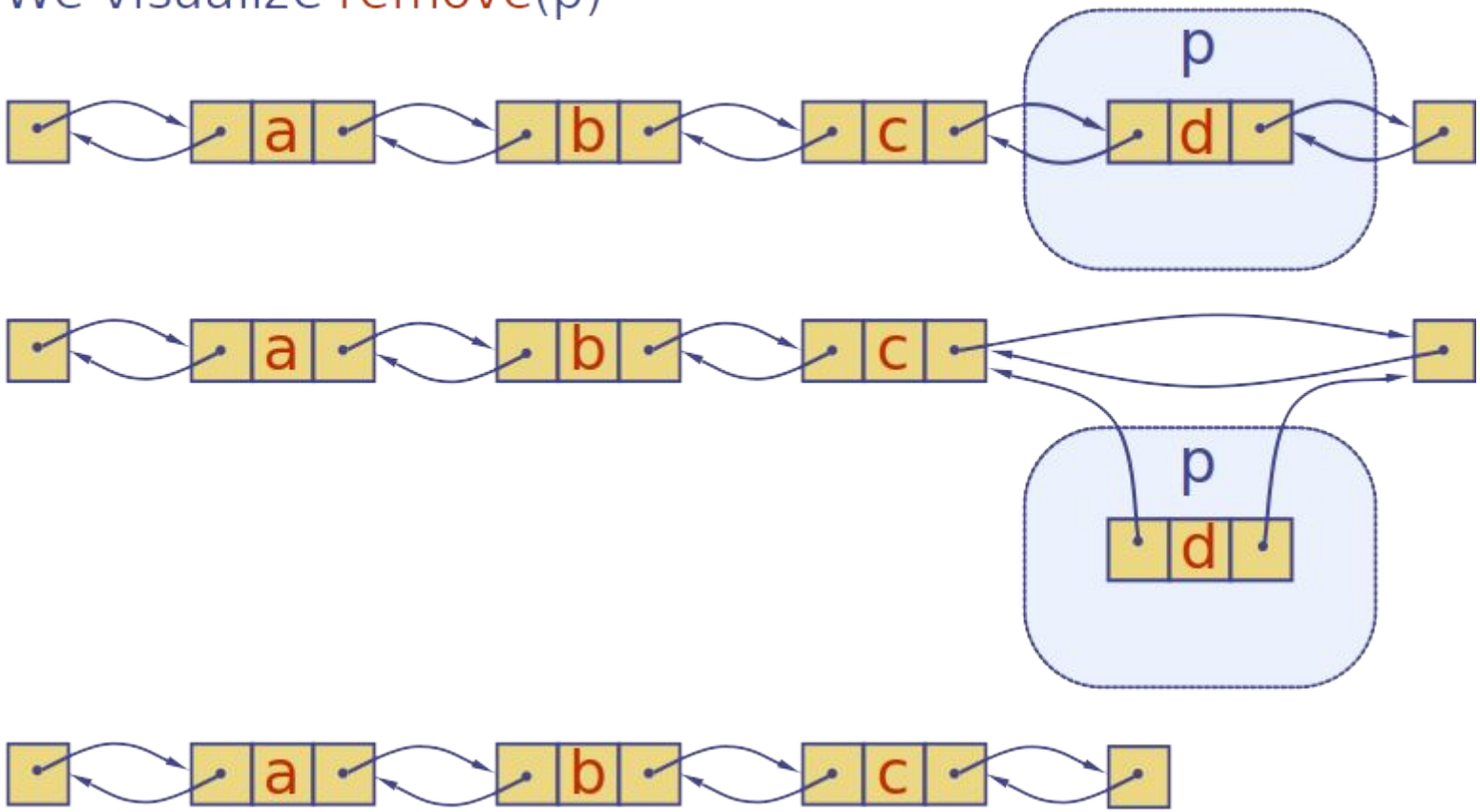


Insertion Algorithm

```
Algorithm insert(p, e):          // insert e before p
    v ←new Node()                // Create a new node v
    v.element ← e                // Insert new element
    u ←p.prev
    v.next ←p; p.prev ←v         // Link in v before p.
    v.prev ←u; u.next ←v         // Link in v after u.
```

Deletion from Doubly Linked List

- We visualize remove(p)



Deletion Algorithm

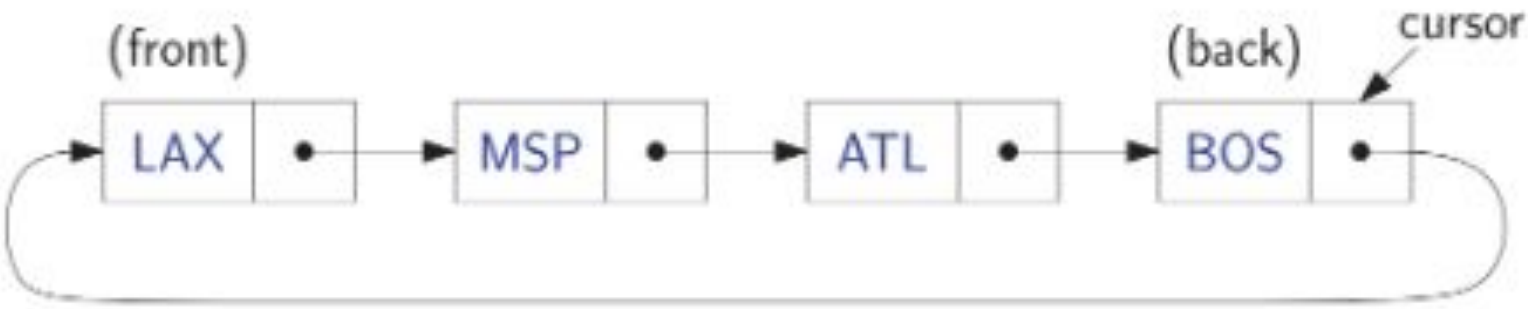
Algorithm *remove*(p):

```
u ← p.prev
w ← p.next
u.next←w                    // Linking out p
w.prev ←u
delete (p)                  // Delete the node pointed to by p.
                            // Neither required nor possible in Java.
```

Complexities

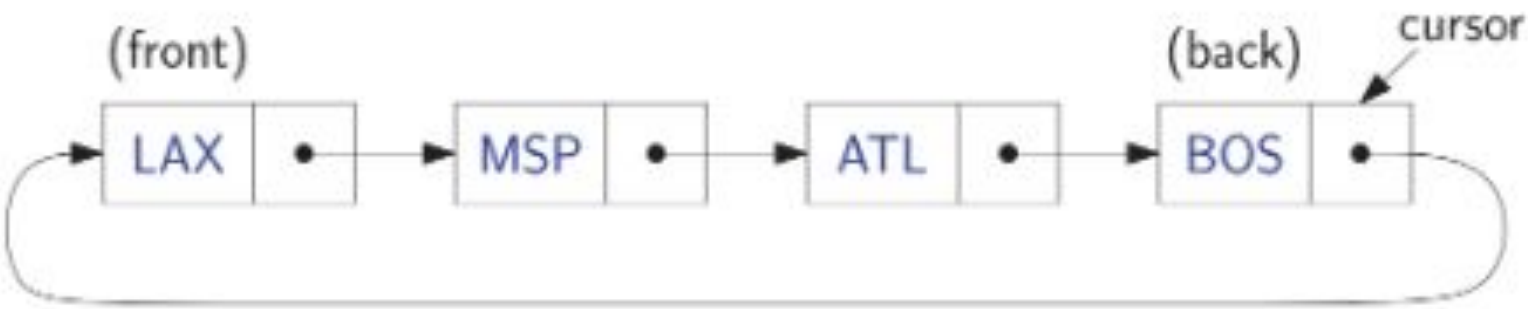
- In the implementation of the doubly linked list:
  - The space used by each position of the list is O(1)
  - The space used by a list with n elements is O(n)
  - All the operations run in O(1) time

Circular Linked List



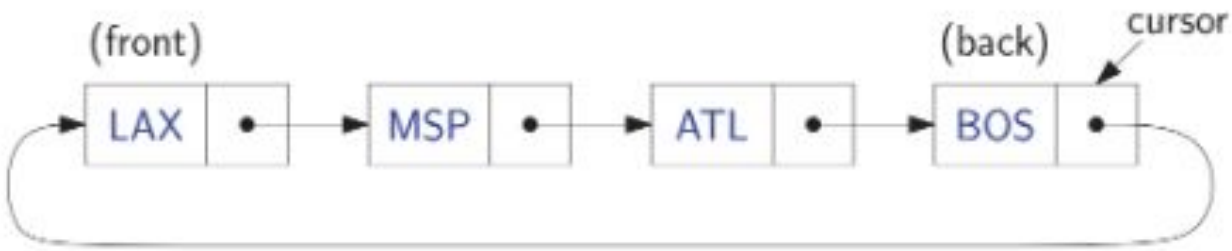
- A ***circularly linked list*** has the same kind of nodes as a singly linked list.
  - That is, each node in a circularly linked list has a next pointer and an element value.
  -
- But, rather than having a head or tail, the nodes of a circularly linked list are linked into a cycle.
- If we traverse the nodes of a circularly linked list from any node by following next pointers, we eventually visit all the nodes and cycle back to the node from which we started.

Circular Linked List



- We need some node to be marked as a special node, which we call the ***cursor***. The cursor node allows us to have a place to start from if we ever need to traverse a circularly linked list.
- There are two positions of particular interest in a circular list.
  - The first is the element that is referenced by the cursor, which is called the ***back***, and
  - the element immediately following this in the circular order, which is called the ***front***.

Circular Linked List Operations



- We define the following functions for a circularly linked list:
  - ***front()***: Return the element referenced by the cursor; an error results if the list is empty.
  - ***back()***: Return the element immediately after the cursor; an error results if the list is empty.
  - ***advance()***: Advance the cursor to the next node in the list.
  - ***add(e)***: Insert a new node with element e immediately after the cursor; if the list is empty, then this node becomes the cursor and its next pointer points to itself.
  - ***remove()***: Remove the node immediately after the cursor (not the cursor itself, unless it is the only node); if the list becomes empty, the cursor is set to null.

Complexities

Operation	Time Complexity	Explanation
`front()`	O(1)	Access the node immediately after the cursor.
`back()`	O(1)	Access the cursor node directly.
`advance()`	O(1)	Move the cursor to the next node.
`add()`	O(1)	Insert a new node after the cursor.
`remove()`	O(1)	Remove the node immediately after the cursor (the front).

Example Java Application:
Maintaining a Playlist for a Digital Audio Player



Circular Linked List data structure in Java

https://github.com/ahmetfeyziates/Algorithms-and-DataStructures-Bootcamp/blob/main/src/CircleList.java

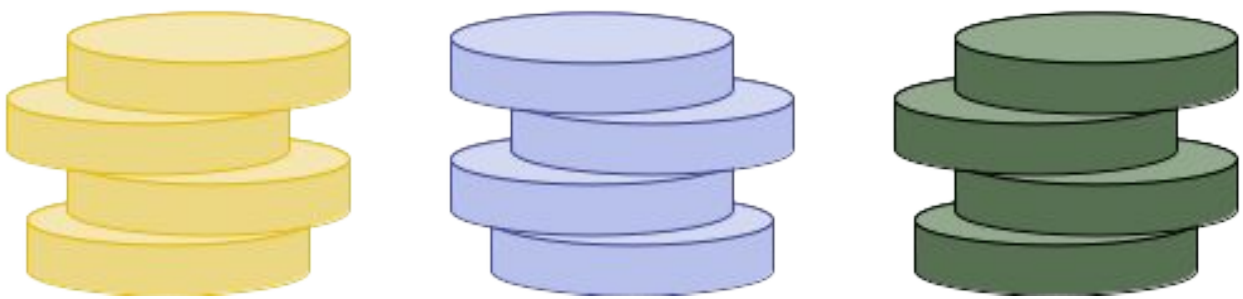
A PlayList for a digital audio player

https://github.com/ahmetfeyziates/Algorithms-and-DataStructures-Bootcamp/blob/main/src/PlayList.java

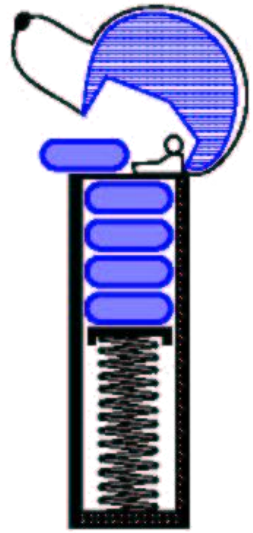
Animations

https://vizlearn.app/algorithms/data-structures/array-list

Stacks



The Stack ADT



- The Stack ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Think of a spring-loaded plate dispenser
- Main stack operations:
  - push(object): inserts an element
  - object pop(): removes the last inserted element
- Auxiliary stack operations:
  - object top(): returns the last inserted element without removing it
  - integer size(): returns the number of elements stored
  - boolean empty(): indicates whether no elements are stored

Stack Interface in Java

Stack of ints

```
public interface Stack {
    int size();

    boolean empty();

    int top() throws StackEmptyException;

    void push(int element);

    void pop() throws StackEmptyException;
}

public class StackEmptyException extends RuntimeException {
    public StackEmptyException(String message) {
        super(message);
    }
}
```

A Generic Stack

```
public interface Stack<E> {
    int size();

    boolean empty();

    E top() throws StackEmptyException;

    void push(E element);

    void pop() throws StackEmptyException;
}

public class StackEmptyException extends RuntimeException {
    public StackEmptyException(String message) {
        super(message);
    }
}
```

Applications of Stacks

- Direct applications
  - Page-visited history in a Web browser
  - Undo sequence in a text editor
  - Chain of method calls in the C++ run-time system
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

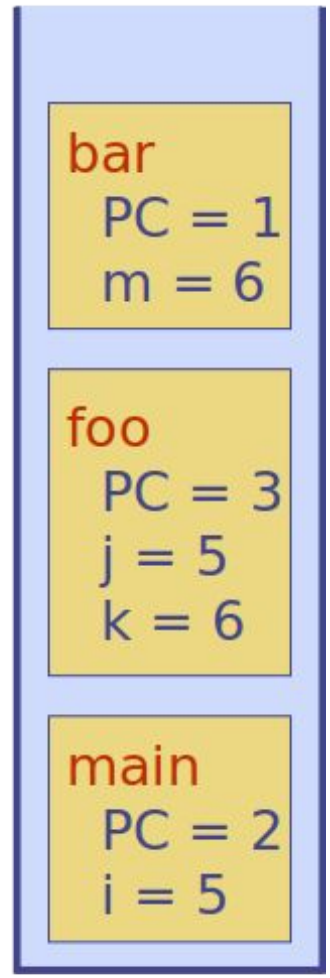
Run-time Stack

- The run-time system keeps track of the chain of active functions with a stack
- When a function is called, the system pushes on the stack a frame containing
  - Local variables and return value
  - Program counter, keeping track of the statement being executed
- When the function ends, its frame is popped from the stack and control is passed to the function on top of the stack
- Allows for recursion

```
main() {
  int i = 5;
  foo(i);
  }

foo(int j) {
  int k;
  k = j+1;
  bar(k);
  }

bar(int m) {
  …
  }
```



Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

```
Algorithm size()
  return t + 1
Algorithm empty()
  return (t < 0)
Algorithm top()
  if empty() then
    throw StackEmptyException
  return S[t]
Algorithm pop()
  if empty() then
    throw StackEmptyException
  else
    t ← t - 1
    return S[t + 1]
```



Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then throw a StackFull exception
  - Limitation of the array-based implementation
  - Not intrinsic to the Stack ADT

Algorithm *push*(o)
  if *size*$() = N$ **then**
    throw *StackFull*
  else
    $t \leftarrow t + 1$
    $S[t] \leftarrow o$



Performance and Limitations

Operation	*Time*
size	$O(1)$
empty	$O(1)$
top	$O(1)$
push	$O(1)$
pop	$O(1)$

- Performance
  - Let n be the number of elements in the stack
  - The space used is $O(N)$, independent from the number t ≤ N of elements that are actually in the stack.
  - Each operation runs in time $O(1)$
- Limitations
  - The maximum size of the stack must be defined a priori and cannot be changed
  - Trying to push a new element into a full stack causes an implementation-specific exception

Array-based Stack Example in Java

https://github.com/ahmetfeyziates/Algorithms-and-DataStructures-Bootcamp/blob/main/src/ArrayStack.java

Queues



The Queue ADT

- The Queue ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
  - enqueue(object): inserts an element at the end of the queue
  - dequeue(): removes the element at the front of the queue
- Auxiliary queue operations:
  - object front(): returns the element at the front without removing it
  - integer size(): returns the number of elements stored
  - boolean empty(): indicates whether no elements are stored
- Exceptions
  - Attempting the execution of dequeue or front on an empty queue throws an QueueEmpty